

Aritmética del Computador

El sistema de punto flotante y sus consecuencias ocultas

Autor: Dr. Hermes Pantoja Carhuavilca

Curso: Métodos Numéricos



$$0.1 \approx 0.000110011001100\dots_2$$

$$\gg 0.1 + 0.2$$

$$0.3000000000000000004$$

$$(-1)^S \times 1.M \times 2^{E-\text{sesgo}}$$

Empecemos con una Simple Pregunta: ¿Cuánto es $0.1 + 0.2$?

```
»> 0.1 + 0.2
```

```
0.30000000000000004
```

```
»> 0.1 + 0.2 == 0.3
```

```
False
```

Confiamos en las computadoras para cálculos de alta precisión. Sin embargo, **fallan en la aritmética más básica.**

Esto no es un error aislado: es una **característica fundamental** de su diseño.

¿Por qué ocurre esto y qué implicaciones tiene para la ingeniería?

El Origen del Problema: Mundos Diferentes, Lenguajes Diferentes

Humanos



Sistema **decimal** (base 10). Precisión teóricamente infinita: números como π o $1/3$ se manejan conceptualmente sin pérdida.

Computadoras

0	1	0	1	1	0	0	1	1	0	1	0
---	---	---	---	---	---	---	---	---	---	---	---

Sistema **binario** (base 2). Memoria y precisión **finitas**: todo número debe caber en un espacio predefinido (p. ej. 32 o 64 bits).

Para representar un rango inmenso de números en un espacio finito, las computadoras usan una aproximación ingeniosa pero imperfecta: la **notación científica binaria**, o **punto flotante**.

La “Traducción” Imposible: Por qué 0.1 no es 0.1

En base 10, $1/3 = 0.333\dots$ es infinito. En base 2 ocurre lo mismo con fracciones “simples” en base 10.

Multiplicaciones sucesivas por 2:

$$0.1 \times 2 = 0.2 \rightarrow 0$$

$$0.2 \times 2 = 0.4 \rightarrow 0$$

$$0.4 \times 2 = 0.8 \rightarrow 0$$

$$0.8 \times 2 = 1.6 \rightarrow 1$$

$$0.6 \times 2 = 1.2 \rightarrow 1$$

$$0.2 \times 2 = 0.4 \rightarrow 0 \text{ (se repite)}$$

$$0.1_{10} =$$

$$0.0 \text{ 0011 0011 0011 } \dots_2$$

El patrón “0011” se repite infinitamente.

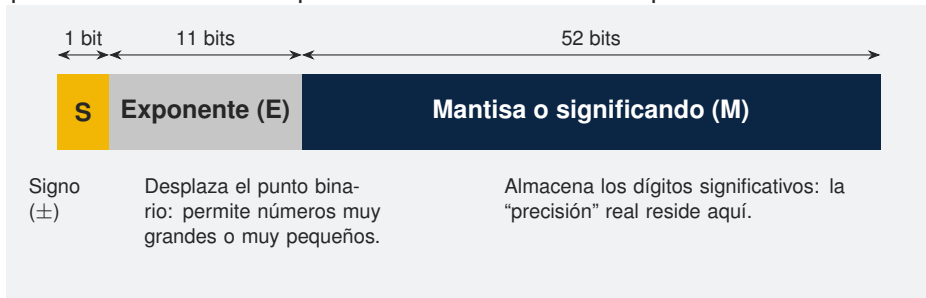
Valor almacenado en doble precisión:

$$0.100000000000000000 \text{ 0555 } \dots$$

La computadora no puede almacenar una secuencia infinita de bits: **debe cortar la representación**. Este es el primer e inevitable error. El número almacenado no es 0.1, sino su aproximación binaria más cercana.

La Arquitectura de los Números: El Estándar IEEE 754

Para garantizar la consistencia entre máquinas, la industria adoptó el estándar **IEEE 754**, que define la estructura para almacenar números de punto flotante.



Fórmula general (números normalizados)

$$x = (-1)^S \times (1.M)_2 \times 2^{E-\text{sesgo}}$$

Sesgo: 127 en precisión simple, 1023 en precisión doble.

Los Dos Formatos Clave: Precisión Simple vs. Doble

Precisión simple (32 bits) — float



Precisión doble (64 bits) — double



	Simple	Doble
Dígitos decimales significativos	≈ 7	$\approx 15-16$
Máximo representable	3.4×10^{38}	1.8×10^{308}
Uso típico	gráficos, GPU	Python, MATLAB, R, C

Concepto clave: normalización. Los números se almacenan de forma que el primer dígito sea siempre 1 (1.xxxxx...). Ese 1 es **implícito** (no se almacena), ganando un bit: 24 bits efectivos (simple) y 53 (doble).

El Límite de la Percepción: Épsilon de Máquina (ϵ)

Definición conceptual: por la precisión finita existe un “salto” entre un número representable y el siguiente. El épsilon de máquina es la distancia entre 1.0 y el siguiente número representable.

Definición formal: $\epsilon = 2^{-(p-1)}$, con p bits de mantisa (incluido el implícito).

Valores típicos:

Simple: $\epsilon = 2^{-23} \approx 1.19 \times 10^{-7}$

Doble: $\epsilon = 2^{-52} \approx 2.22 \times 10^{-16}$



En doble precisión:

`1 + 2**-53 == 1` → **True**

Cualquier contribución más pequeña que ϵ (relativa a la magnitud) se pierde: es el “**ruido de fondo**” del cómputo numérico. El error relativo de redondeo está acotado por $\epsilon/2$.

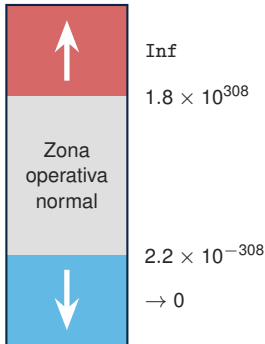
Los Límites del Universo Numérico: Overflow y Underflow

Overflow (desbordamiento)

Qué es: el resultado supera el máximo representable ($\approx 1.8 \times 10^{308}$ en doble).

Resultado: valor especial Inf.

Peligro: Inf se propaga por los cálculos ($\text{Inf} - \text{Inf} = \text{NaN}$), invalidando el resultado final.



Underflow

Qué es: el resultado es más cercano a cero que el mínimo positivo representable ($\approx 2.2 \times 10^{-308}$ normalizado; $\approx 4.9 \times 10^{-324}$ subnormal).

Resultado: se redondea a cero.

Peligro: pérdida sutil pero catastrófica en algoritmos con valores muy pequeños (probabilidades, gradientes).

El Error Más Peligroso: Cancelación por Sustracción

El problema

¿Qué sucede cuando restamos dos números de punto flotante muy cercanos?

Explicación

Los dígitos principales, que contenían la información precisa, se cancelan. Los que quedan son esencialmente “ruido” magnificado, proveniente de los errores de redondeo previos en x e y . **Se pierde casi toda la precisión útil.**

Esto es fatal en algoritmos de control, balística o cualquier sistema que dependa de la diferenciación numérica de señales ruidosas.

$$\begin{array}{r} x = 0.1234567891234567 \\ y = 0.1234567891234512 \\ \hline x - y = 0.0000000000000055 \end{array}$$

Exacto: 5.5×10^{-15}

En doble precisión: $5.4956 \dots e-15$

Solo 2 cifras correctas (error $\approx 0.08\%$),
aunque x e y tenían 16.

Un Ejemplo en el Código: La Raíz de una Ecuación Cuadrática

Contexto: resolver $ax^2 + bx + c = 0$ con $x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$. **Peligro:** si $b^2 \gg 4ac$, entonces $\sqrt{b^2 - 4ac} \approx |b|$ y $-b + \sqrt{b^2 - 4ac}$ es una resta de números casi iguales.

Ingenuo y peligroso ✗

```
% b>0 grande: x1 sufre cancelacion
d = sqrt(b^2 - 4*a*c);
x1 = (-b + d) / (2*a);
x2 = (-b - d) / (2*a);
```

Numéricamente estable ✓

```
% Evita restar numeros casi iguales
d = sqrt(b^2 - 4*a*c);
q = -0.5*(b + sign(b)*d);
x1 = q / a;
x2 = c / q; % Vieta: x1*x2 = c/a
```

$x^2 + 10^8x + 1 = 0$ (raíz pequeña)	Resultado	Error relativo
Fórmula ingenua	-7.4506×10^{-9}	25.5 %
Fórmula estable	-1.0000×10^{-8}	$< 10^{-15}$

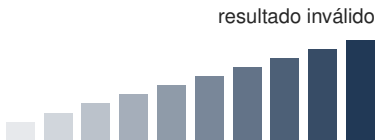
Lección: conocer la aritmética del computador permite elegir un algoritmo matemáticamente equiv-alente pero numéricamente superior.

El Efecto Dominó: Acumulación de Errores

Un solo error de redondeo es minúsculo. Pero, ¿qué pasa tras miles de millones de operaciones?

Caso: simulaciones climáticas

Modelos que simulan el clima durante décadas. Cada paso de tiempo introduce un pequeño error de redondeo. Tras millones de pasos, el error acumulado puede superar la señal que se mide.



Caso: sumatorias de larga duración

Al sumar términos pequeños a un total grande, su contribución puede perderse.

Ejemplo (precisión simple): $\sum_{k=1}^{10^6} \frac{1}{k^2}$

Exacto 1.6449331

De mayor a menor 1.6447253

De menor a mayor 1.6449330

Solución: sumar de menor a mayor (o suma de Kahan).

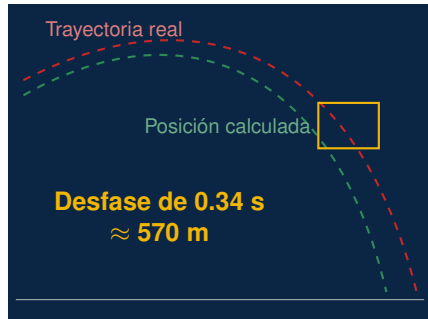
Caso de Estudio: El Fallo del Sistema Patriot

Guerra del Golfo, 1991

Contexto: el sistema de defensa Patriot calculaba la posición de misiles Scud entrantes a partir del tiempo.

El fallo técnico: el reloj interno contaba el tiempo en décimas de segundo. El número $1/10$ tiene representación binaria infinita; el sistema usaba registros de **24 bits** y la truncaba, con un error de $\approx 9.5 \times 10^{-8}$ s por tic.

La acumulación: tras 100 horas de operación (3.6×10^6 tics): $3.6 \times 10^6 \times 9.5 \times 10^{-8} \approx \mathbf{0.34\ s}$



La consecuencia: la ventana de seguimiento quedó desplazada y el Scud no fue interceptado. Impactó un cuartel en Dhahran (25 de febrero de 1991), causando **28 muertes**.

La Lección del Patriot: La Precisión no es un Detalle Académico

Análisis del fallo

- **Error:** no era un “bug” en el sentido tradicional; el código hacía lo que se le pedía.
- **Causa raíz:** falta de comprensión sobre cómo las computadoras representan y redondean los números.
- **El costo:** un error de truncamiento de $\approx 10^{-7}$ s por tic se acumuló hasta convertirse en una catástrofe.

*Este es el caso de estudio definitivo sobre la importancia crítica de la aritmética del computador: la elección de la precisión y el manejo de los **errores de redondeo** son una responsabilidad central del ingeniero, no un problema secundario.*

Resumen de Competencias Adquiridas

Al finalizar esta sesión, usted puede:

- 1 **Explicar** por qué las computadoras usan punto flotante y cómo el estándar IEEE 754 estructura los números reales en un formato binario finito.
- 2 **Identificar** las limitaciones del sistema: errores de redondeo, overflow, underflow y cancelación por sustracción.
- 3 **Calcular y entender** el épsilon de máquina (ϵ) como el límite de resolución del sistema.
- 4 **Predecir** fallos numéricos en algoritmos comunes y reconocer la importancia de la estabilidad numérica.
- 5 **Distinguir** la precisión adecuada (simple vs. doble) para una tarea de ingeniería, priorizando la seguridad y la robustez.

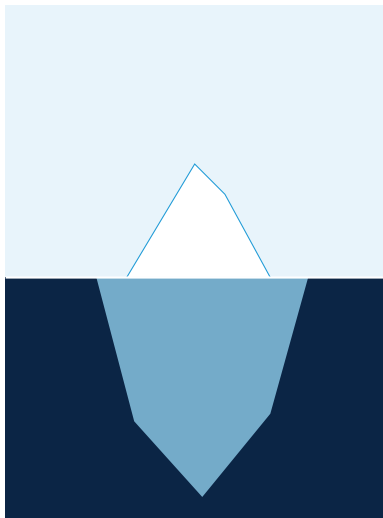
La Responsabilidad del Ingeniero: Más Allá de Escribir Código

El veredicto: la aritmética de punto flotante es una herramienta poderosa pero fundamentalmente imperfecta.



- No basta con saber programar: los números en la computadora son **discretos** y son **aproximaciones**.
- El **conocimiento de la aritmética subyacente** no es opcional: separa un diseño robusto de un fallo catastrófico.
- La **estabilidad numérica** debe ser siempre una prioridad sobre la simple velocidad de cálculo.

El Número que Ves no es el Número que Es



Punta del iceberg (visible)

$0.1 + 0.2 = 0.300000000000000004$: un simple “glitch”.

Masa sumergida (invisible)

El sistema de punto flotante, los errores de redondeo, la cancelación catastrófica, el estándar IEEE 754 y la responsabilidad del ingeniero sobre la vida y la seguridad.

La próxima vez que vea un resultado numérico inesperado, no lo descarte como un error: véalo como un recordatorio de la compleja y frágil maquinaria que sustenta nuestro mundo digital. **Comprenderla es su deber.**

Referencias Bibliográficas

1. Chapra, S. C., & Canale, R. P. (2015). *Métodos Numéricos para Ingenieros*. McGraw-Hill Education.
2. Burden, R. L., & Faires, J. D. (2011). *Análisis Numérico*. Cengage Learning.
3. Goldberg, D. (1991). What every computer scientist should know about floating-point arithmetic. *ACM Computing Surveys*, 23(1), 5–48.
4. U.S. General Accounting Office (1992). *Patriot Missile Defense: Software Problem Led to System Failure at Dhahran, Saudi Arabia* (GAO/IMTEC-92-26).